

# Transformers

---

Carlos Cotrini

*CAS Building ML/AI Applications, ETH Zurich, weekend 2*

*Friday 18 September 2026*

1

*Section 1*

# Multi-headed attention

---

# Multi-headed attention

---

Remember that we need multiple attention maps.

Alice    tried    golf    and    tennis    and    she    liked    them

# Multi-headed attention

---

Remember that we need multiple attention maps.

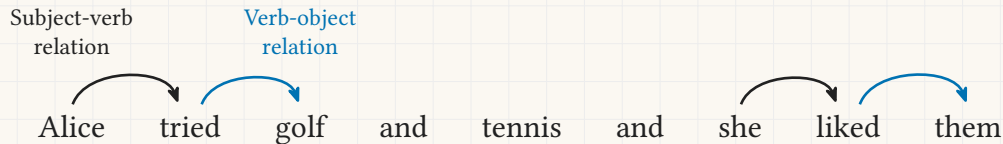
Subject-verb  
relation



# Multi-headed attention

---

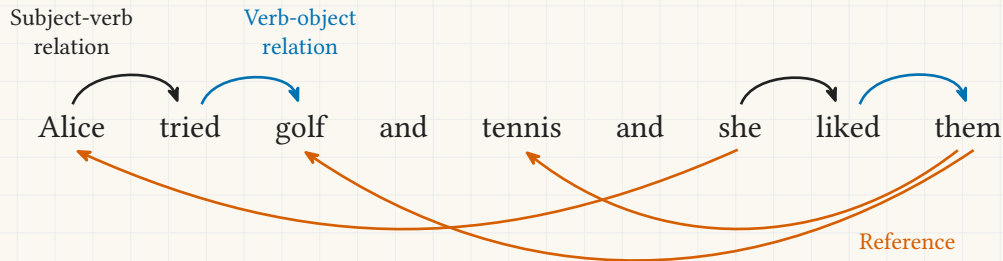
Remember that we need multiple attention maps.



# Multi-headed attention

---

Remember that we need multiple attention maps.



# Multi-headed attention mechanism

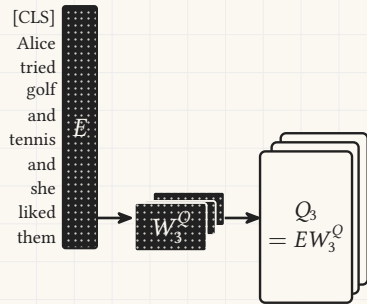
---

[CLS]  
Alice  
tried  
golf  
and  
tennis  
and  
she  
liked  
them



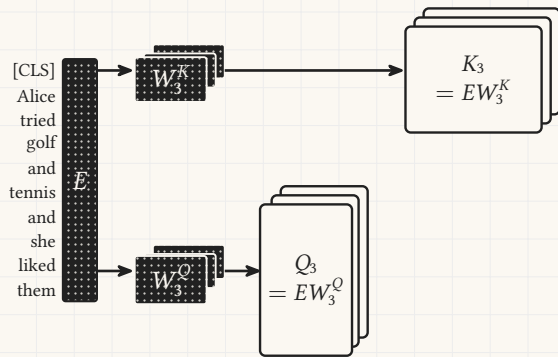
# Multi-headed attention mechanism

---

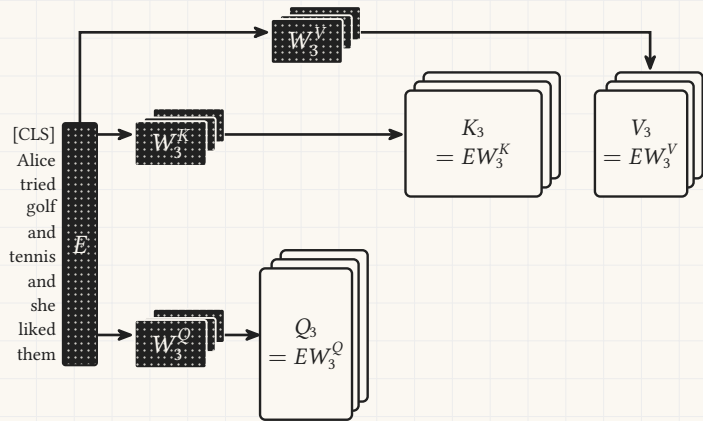


# Multi-headed attention mechanism

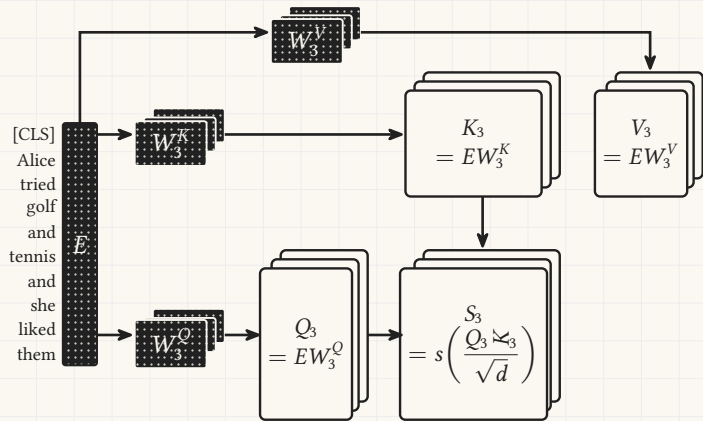
---



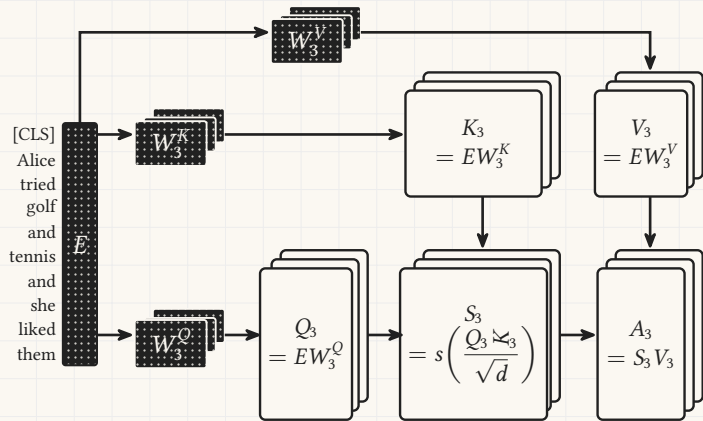
# Multi-headed attention mechanism



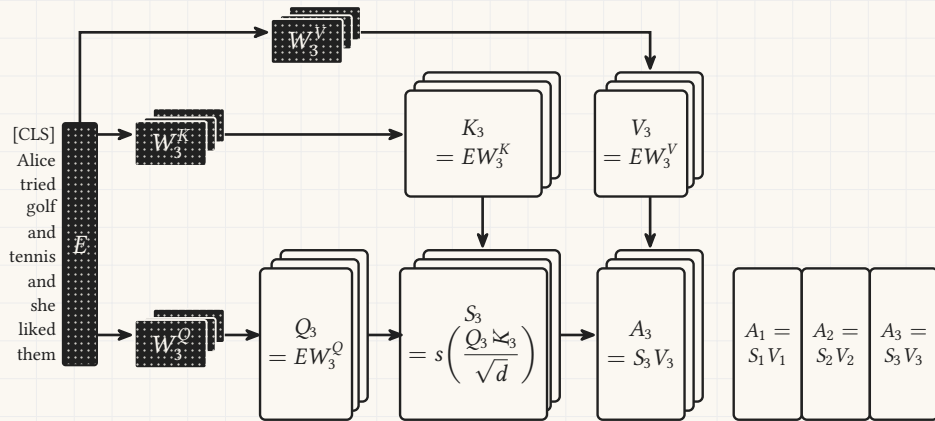
# Multi-headed attention mechanism



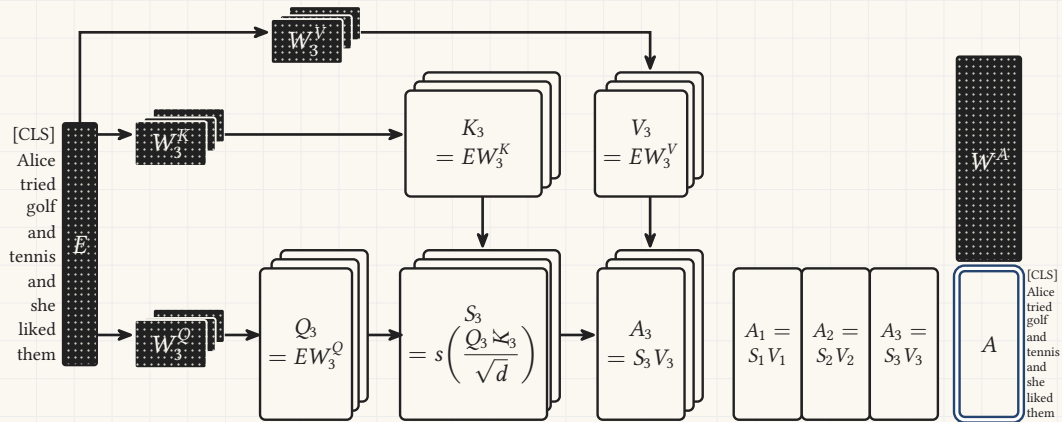
# Multi-headed attention mechanism



# Multi-headed attention mechanism



# Multi-headed attention mechanism



2

*Section 2*

# Cross-attention

---

# Cross-attention

---

- Consider the problem of translating from English to German.

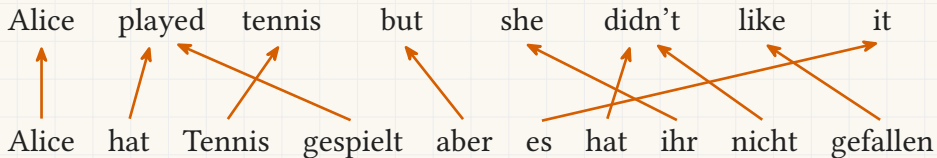
Alice played tennis but she didn't like it

Alice hat Tennis gespielt aber es hat ihr nicht gefallen

# Cross-attention

---

- Consider the problem of translating from English to German.
- A different attention mechanism is needed.



# Abstraction of a cross-attention mechanism

---

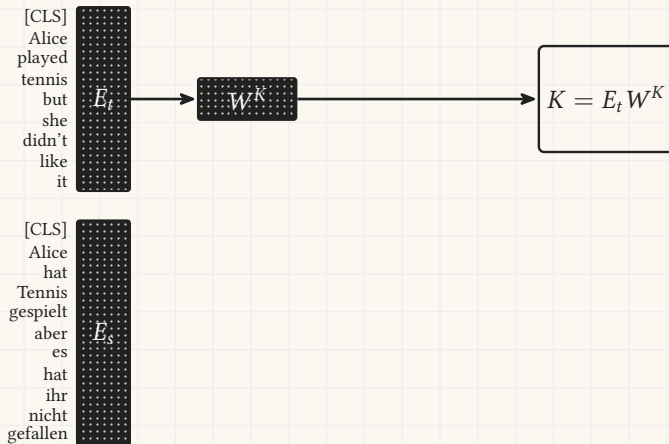
[CLS]  
Alice  
played  
tennis  
but  
she  
didn't  
like  
it

$E_t$

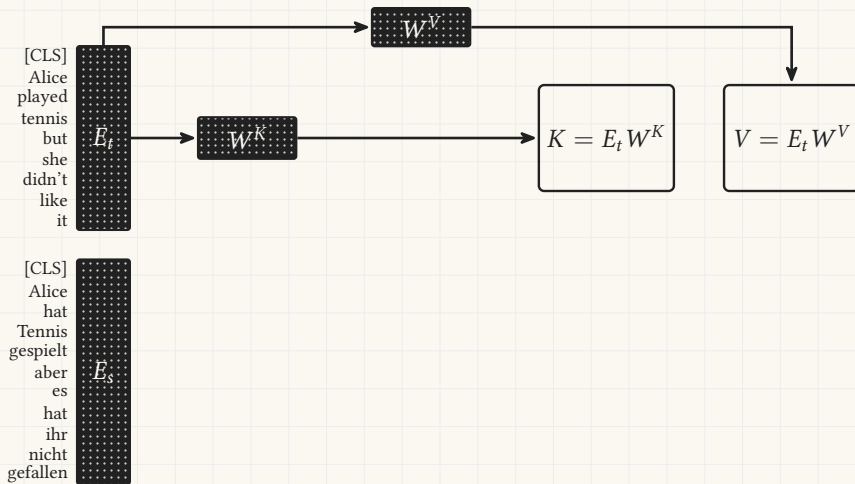
[CLS]  
Alice  
hat  
Tennis  
gespielt  
aber  
es  
hat  
ihr  
nicht  
gefallen

$E_s$

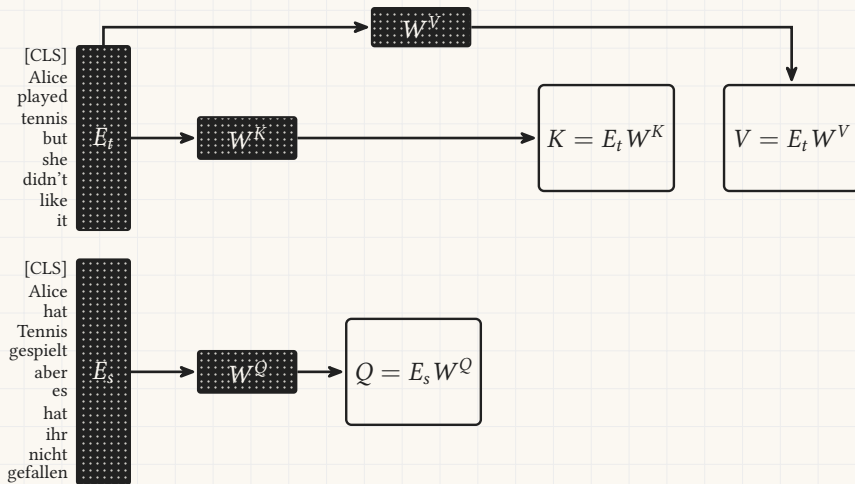
# Abstraction of a cross-attention mechanism



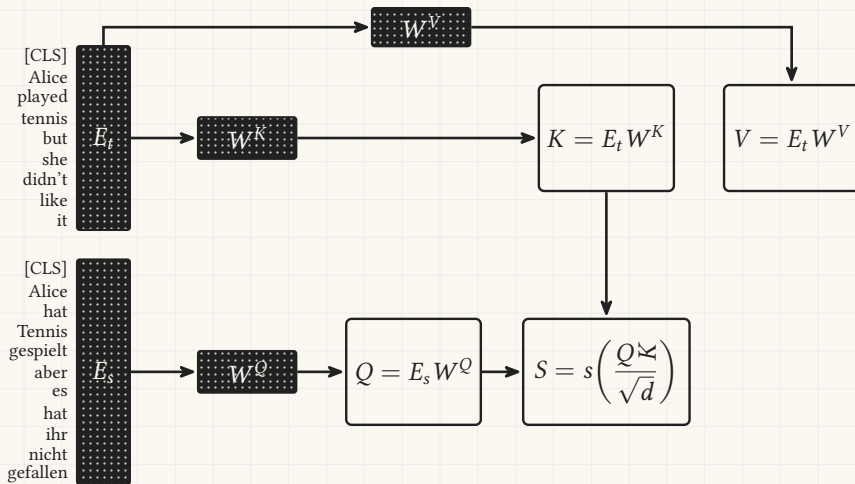
# Abstraction of a cross-attention mechanism



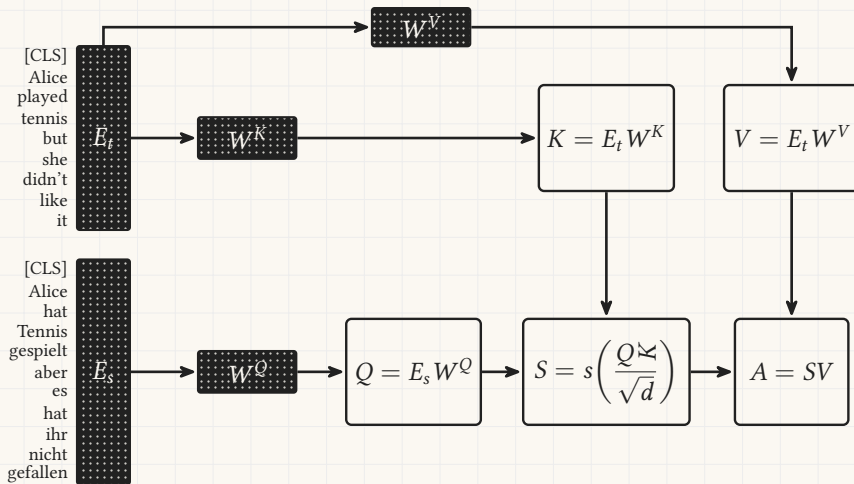
# Abstraction of a cross-attention mechanism



# Abstraction of a cross-attention mechanism



# Abstraction of a cross-attention mechanism



3

*Section 3*

# Masked self-attention

---

# Masked self-attention

---

- Consider the task of generating text for answering questions.

Why        did                the        bicycle        fall                over

# Masked self-attention

---

- Consider the task of generating text for answering questions.

Why        did            the        bicycle        fall            over

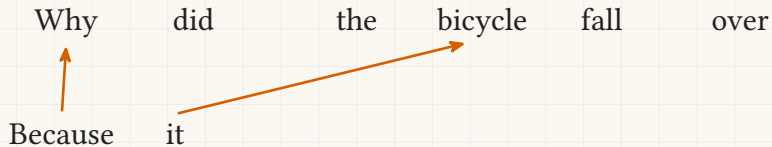
↑

Because

# Masked self-attention

---

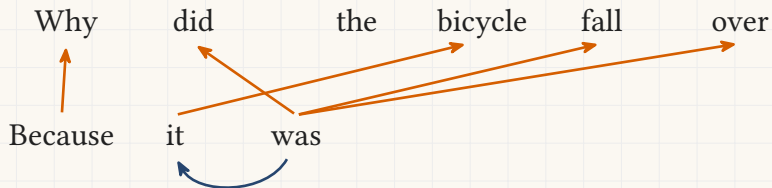
- Consider the task of generating text for answering questions.



# Masked self-attention

---

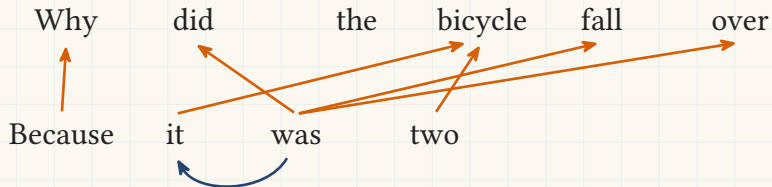
- Consider the task of generating text for answering questions.



# Masked self-attention

---

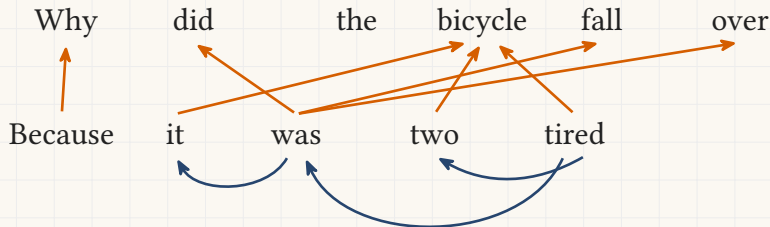
- Consider the task of generating text for answering questions.



# Masked self-attention

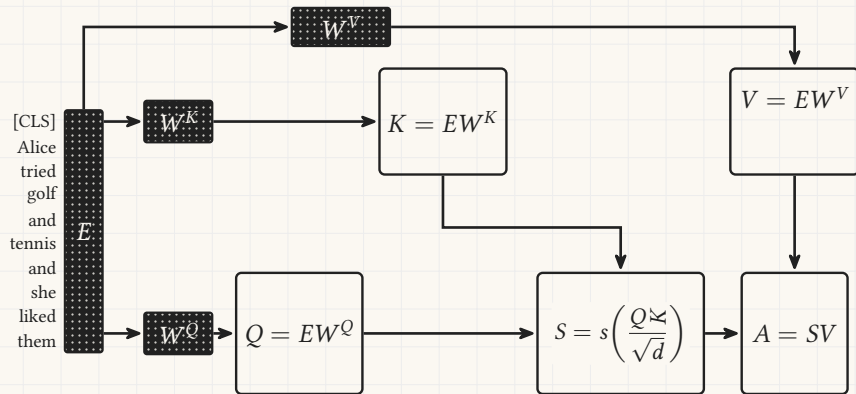
---

- Consider the task of generating text for answering questions.

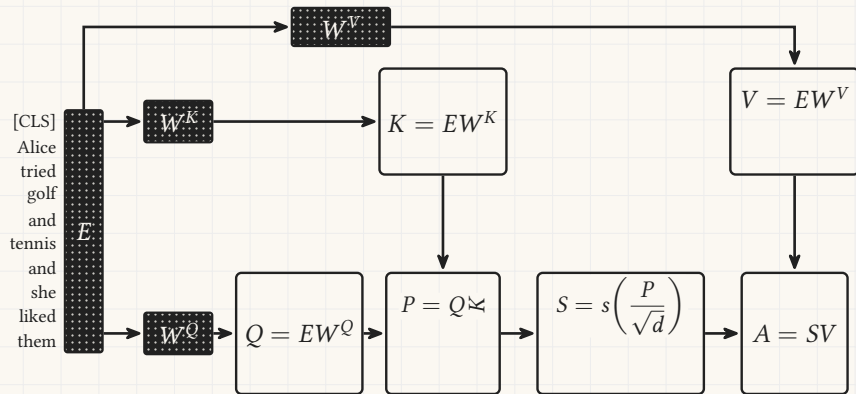


For generating sentences, we need an attention mechanism that only looks at what has been produced so far.

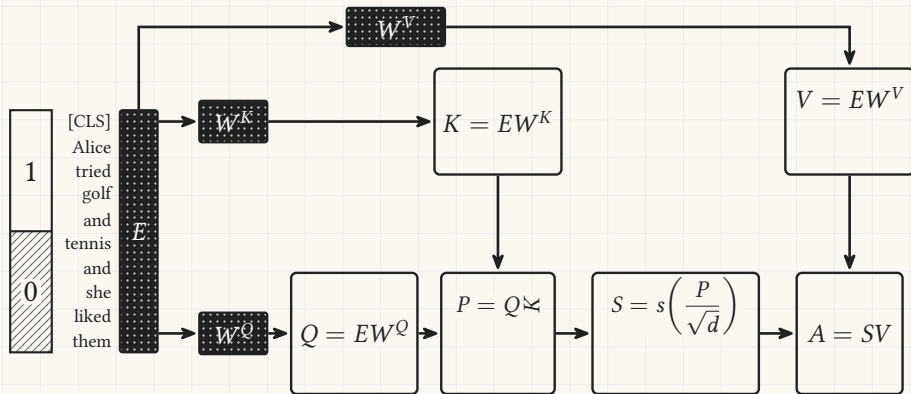
# Masked self-attention



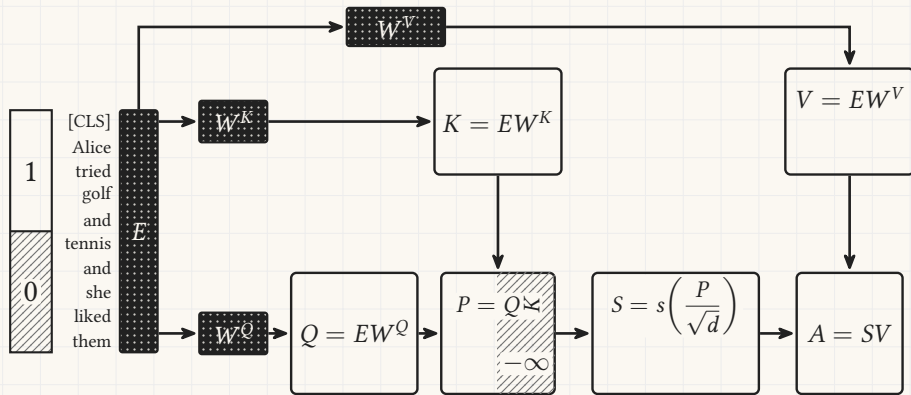
# Masked self-attention



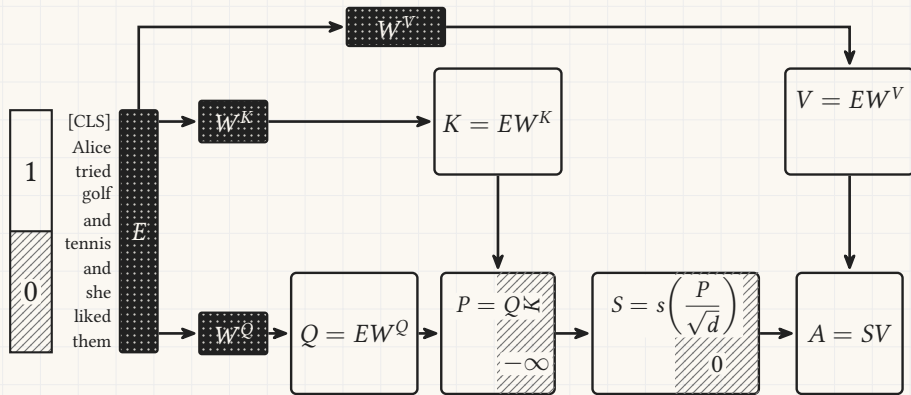
# Masked self-attention



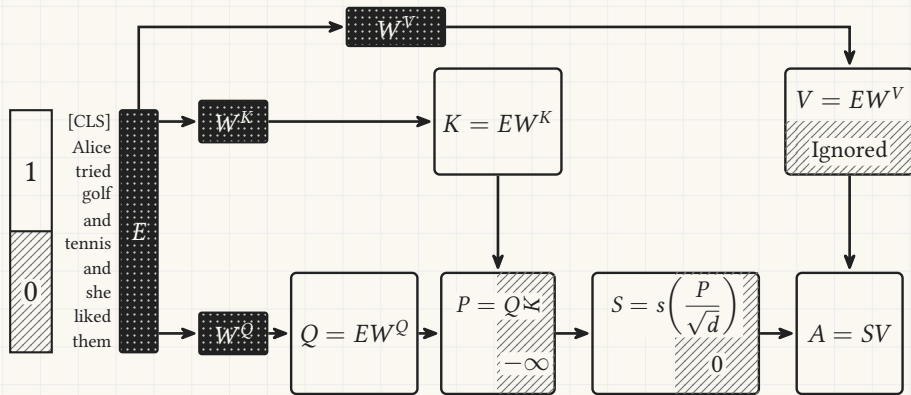
# Masked self-attention



# Masked self-attention



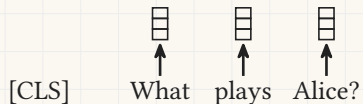
# Masked self-attention



Note how the final embeddings depend only on the embeddings of the unmasked tokens!

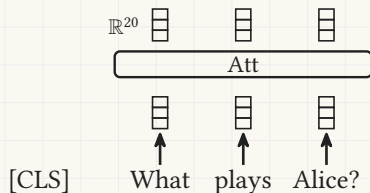
# A simplified transformer architecture

---



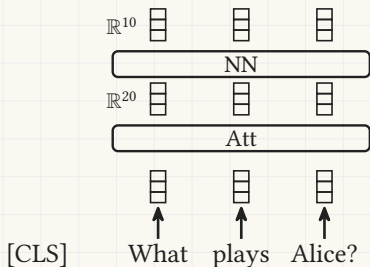
# A simplified transformer architecture

---

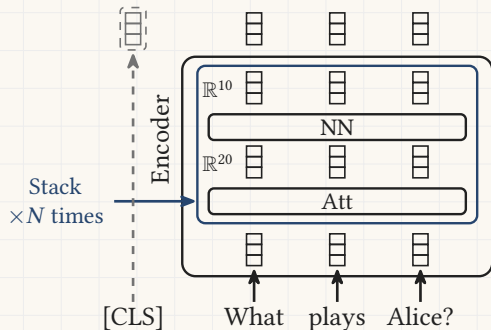


# A simplified transformer architecture

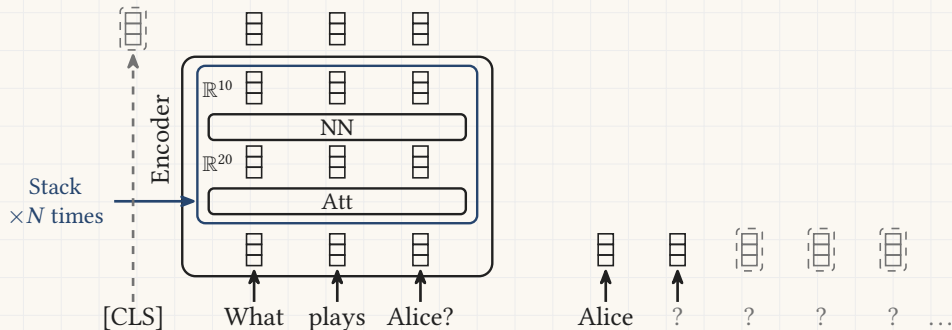
---



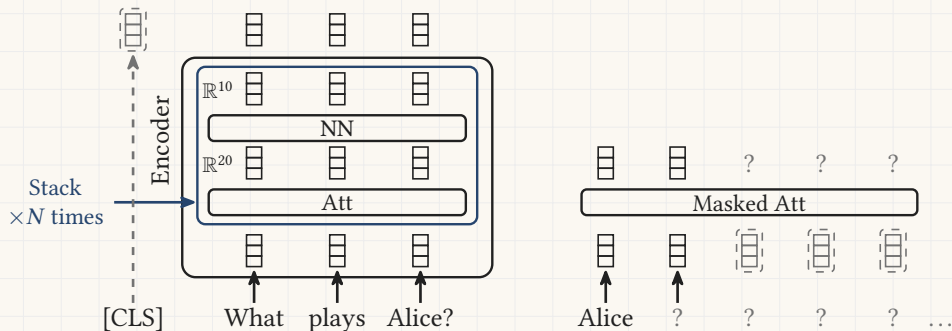
# A simplified transformer architecture



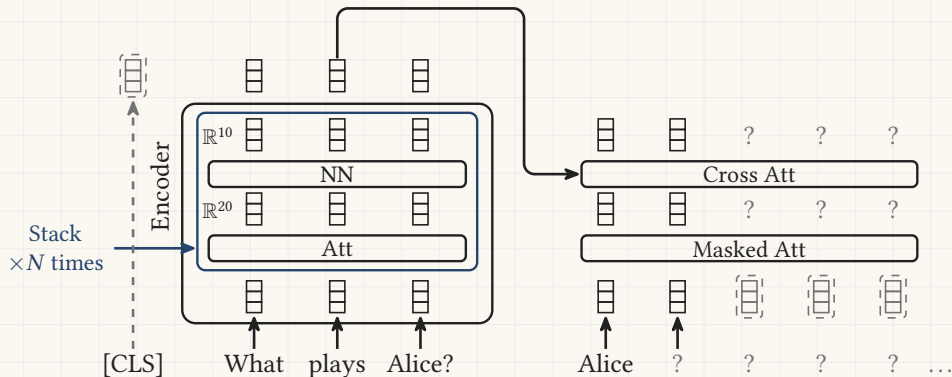
# A simplified transformer architecture



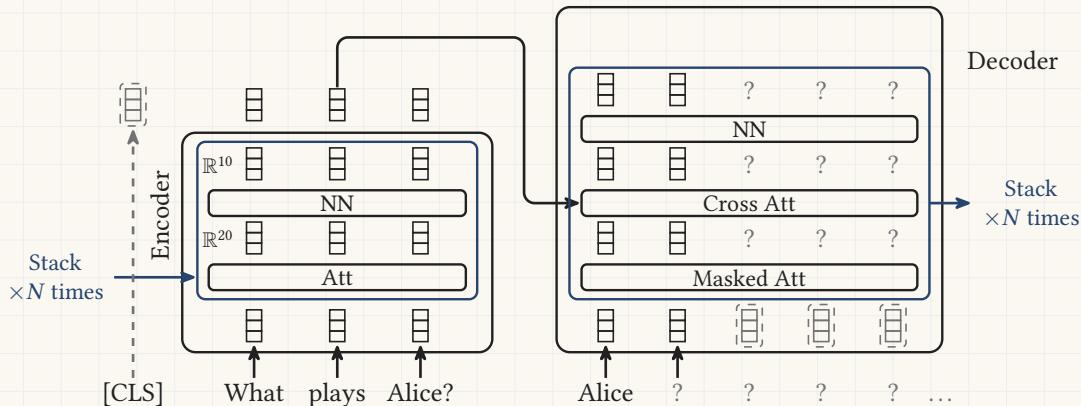
# A simplified transformer architecture



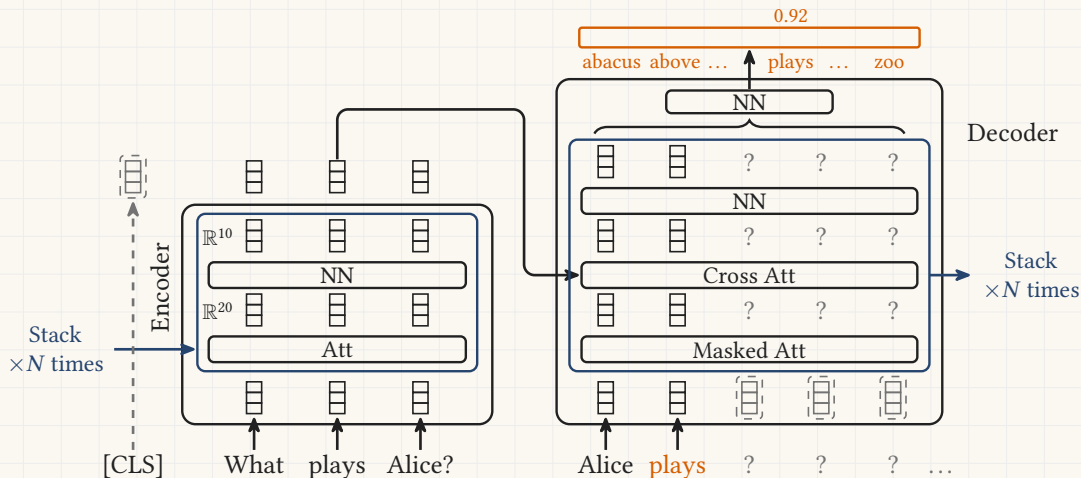
# A simplified transformer architecture



# A simplified transformer architecture



# A simplified transformer architecture



4

*Section 4*

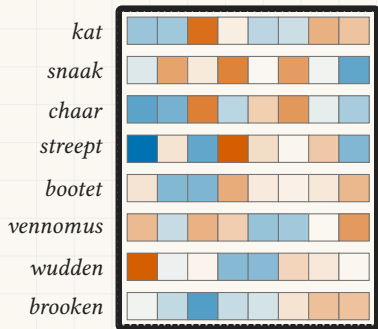
# Training a transformer

---

# The initial embeddings

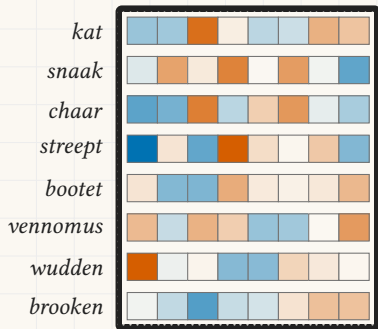
---

One row of numbers per word



# The initial embeddings

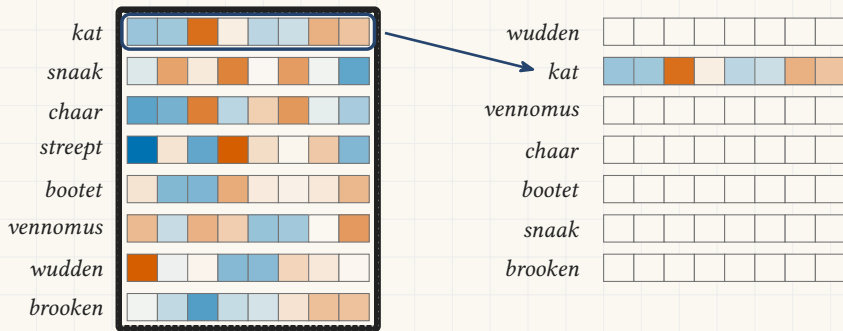
One row of numbers per word



|                 |                                                                                                 |
|-----------------|-------------------------------------------------------------------------------------------------|
| <i>wudden</i>   | <div></div> <div></div> <div></div> <div></div> <div></div> <div></div> <div></div> <div></div> |
| <i>kat</i>      | <div></div> <div></div> <div></div> <div></div> <div></div> <div></div> <div></div> <div></div> |
| <i>vennomus</i> | <div></div> <div></div> <div></div> <div></div> <div></div> <div></div> <div></div> <div></div> |
| <i>chaar</i>    | <div></div> <div></div> <div></div> <div></div> <div></div> <div></div> <div></div> <div></div> |
| <i>bootet</i>   | <div></div> <div></div> <div></div> <div></div> <div></div> <div></div> <div></div> <div></div> |
| <i>snaak</i>    | <div></div> <div></div> <div></div> <div></div> <div></div> <div></div> <div></div> <div></div> |
| <i>brooken</i>  | <div></div> <div></div> <div></div> <div></div> <div></div> <div></div> <div></div> <div></div> |

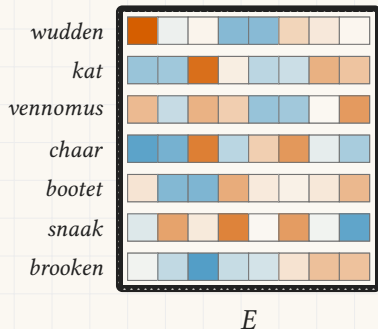
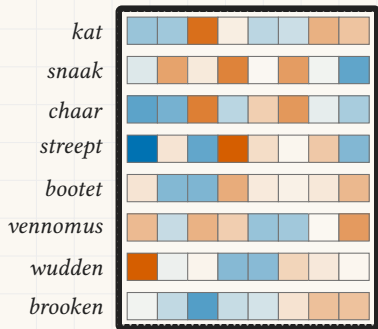
# The initial embeddings

One row of numbers per word



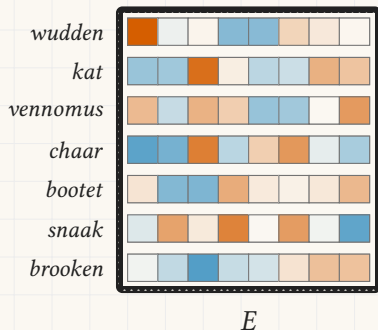
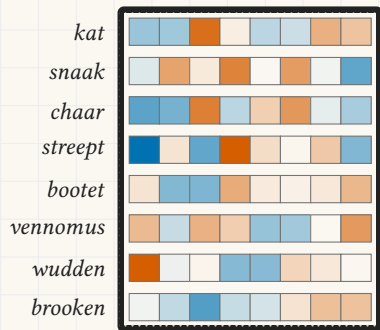
# The initial embeddings

One row of numbers per word



# The initial embeddings

One row of numbers per word



These numbers are weights too, like  $W^Q$ ,  $W^K$  and  $W^V$ . Training guesses them

# The embeddings are guessed in training

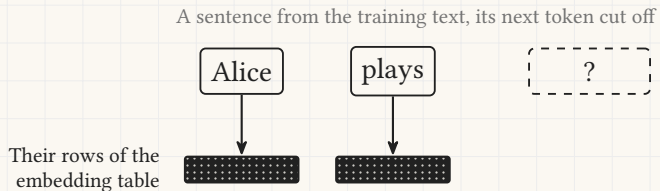
---

A sentence from the training text, its next token cut off

Alice plays ?

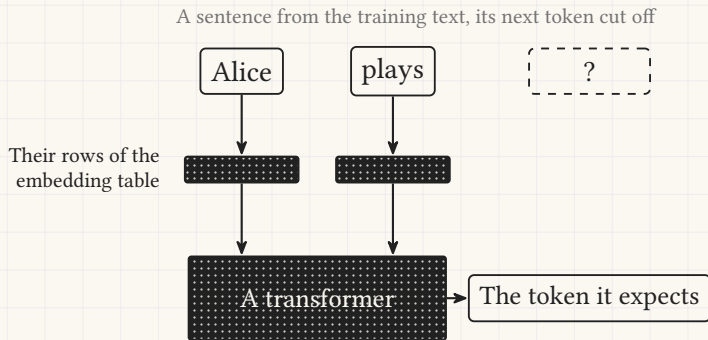
# The embeddings are guessed in training

---



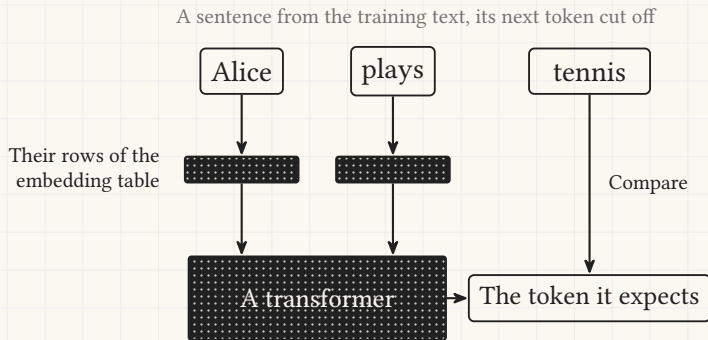
# The embeddings are guessed in training

---



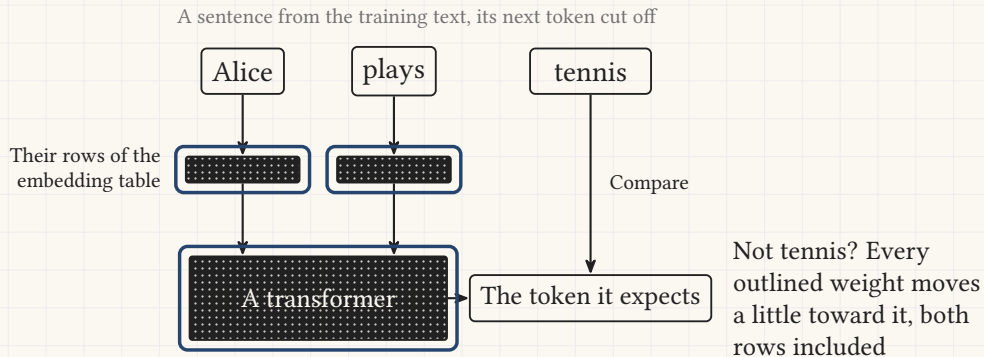
# The embeddings are guessed in training

---



# The embeddings are guessed in training

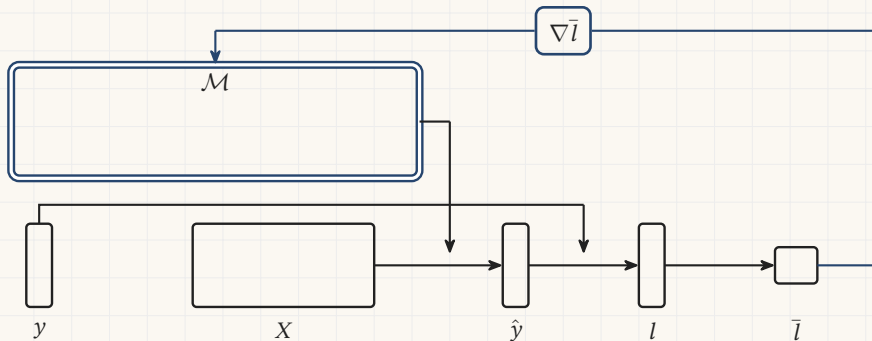
---



# Training a transformer

A row is text with its next token cut off. Every position of every document gives one.

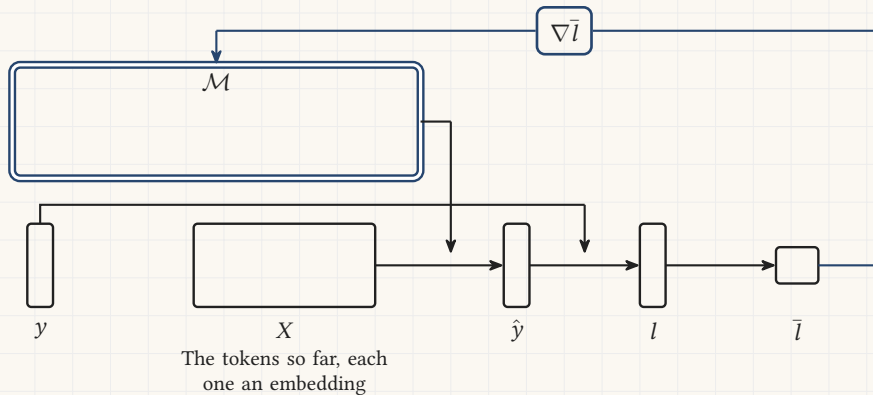
The box holds a transformer. Vaswani and others, *Attention is all you need*, 2017.



# Training a transformer

A row is text with its next token cut off. Every position of every document gives one.

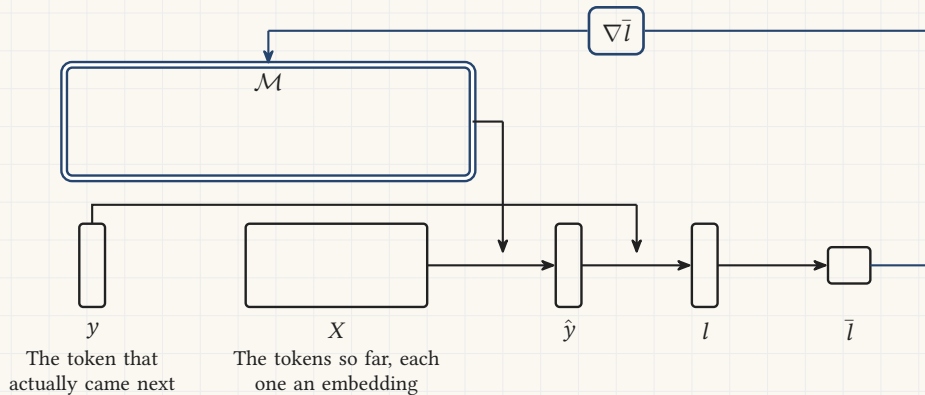
The box holds a transformer. Vaswani and others, *Attention is all you need*, 2017.



# Training a transformer

A row is text with its next token cut off. Every position of every document gives one.

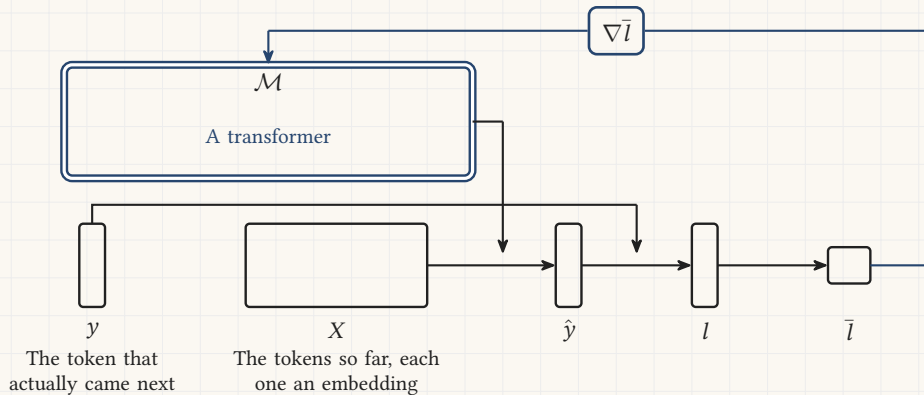
The box holds a transformer. Vaswani and others, *Attention is all you need*, 2017.



# Training a transformer

A row is text with its next token cut off. Every position of every document gives one.

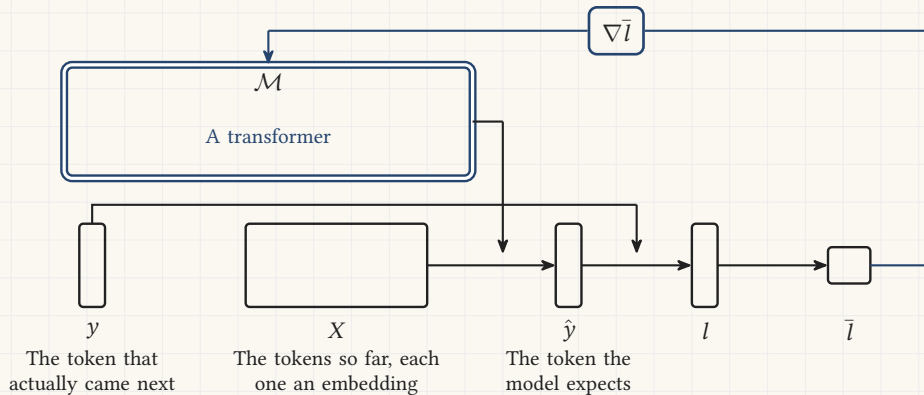
The box holds a transformer. Vaswani and others, *Attention is all you need*, 2017.



# Training a transformer

A row is text with its next token cut off. Every position of every document gives one.

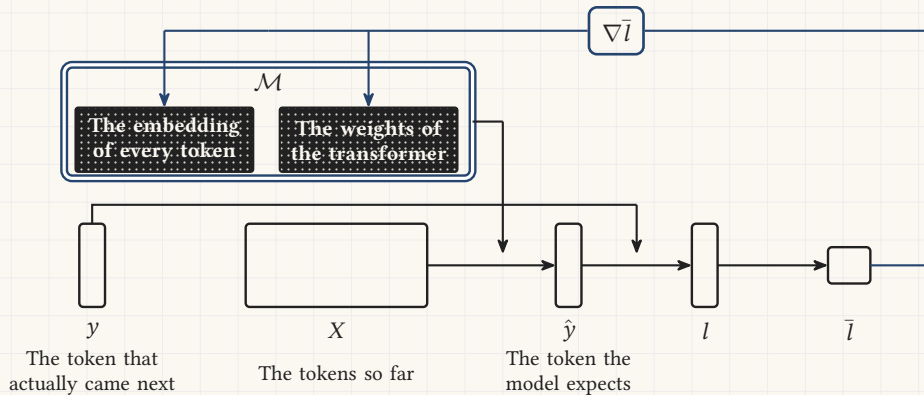
The box holds a transformer. Vaswani and others, *Attention is all you need*, 2017.



# Training a transformer

A row is text with its next token cut off. Every position of every document gives one.

The box holds a transformer. Vaswani and others, *Attention is all you need*, 2017.



5

*Section 5*

# Positional encodings

---

# Positional encodings

---

- Consider the sentences *the dog chased the cat* and *the cat chased the dog*.

# Positional encodings

---

- Consider the sentences *the dog chased the cat* and *the cat chased the dog*.
- They are permutations of each other, so the attention maps will also just be permutations of each other.

# Positional encodings

---

- Consider the sentences *the dog chased the cat* and *the cat chased the dog*.
- They are permutations of each other, so the attention maps will also just be permutations of each other.
- Note that attention mechanisms do not pay attention to the position of a token in the sentence.

# Binary positional encoding

---

|         |   |   |   |   |   |   |
|---------|---|---|---|---|---|---|
| [CLS]   | 0 | 0 | 0 | 0 | 0 | 0 |
| because | 1 | 0 | 0 | 0 | 0 | 0 |
| it      | 0 | 1 | 0 | 0 | 0 | 0 |
| was     | 1 | 1 | 0 | 0 | 0 | 0 |
| two     | 0 | 0 | 1 | 0 | 0 | 0 |
| tired   | 1 | 0 | 1 | 0 | 0 | 0 |
| [EOS]   | 0 | 1 | 1 | 0 | 0 | 0 |

*BP*

# Binary positional encoding

---

|         |   |   |   |   |   |   |
|---------|---|---|---|---|---|---|
| [CLS]   | 0 | 0 | 0 | 0 | 0 | 0 |
| because | 1 | 0 | 0 | 0 | 0 | 0 |
| it      | 0 | 1 | 0 | 0 | 0 | 0 |
| was     | 1 | 1 | 0 | 0 | 0 | 0 |
| two     | 0 | 0 | 1 | 0 | 0 | 0 |
| tired   | 1 | 0 | 1 | 0 | 0 | 0 |
| [EOS]   | 0 | 1 | 1 | 0 | 0 | 0 |

$BP$

$$\begin{aligned}BP_{ij} &= (i \div 2^j) \bmod 2 \\ &= \lfloor 2^{-j} i \rfloor \bmod 2\end{aligned}$$

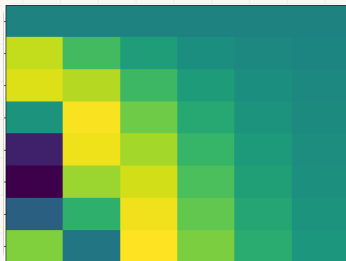
- We would like to create an embedding that encodes the position of the word in the sentence.
- A natural idea is to use a binary encoding of the position.
- Note that the entry  $i, j$  of this positional matrix has the form  $f(\alpha^{-j}i)$ , where  $\alpha > 0$  is a scaling factor and  $f$  is a periodic function.
- In the binary positional encoding case,  $f(x) = \lfloor x \rfloor \bmod 2$ .

# Binary positional encoding

|         |   |   |   |   |   |   |
|---------|---|---|---|---|---|---|
| [CLS]   | 0 | 0 | 0 | 0 | 0 | 0 |
| because | 1 | 0 | 0 | 0 | 0 | 0 |
| it      | 0 | 1 | 0 | 0 | 0 | 0 |
| was     | 1 | 1 | 0 | 0 | 0 | 0 |
| two     | 0 | 0 | 1 | 0 | 0 | 0 |
| tired   | 1 | 0 | 1 | 0 | 0 | 0 |
| [EOS]   | 0 | 1 | 1 | 0 | 0 | 0 |

$BP$

$$\begin{aligned}BP_{ij} &= (i \div 2^j) \bmod 2 \\ &= \lfloor 2^{-j} i \rfloor \bmod 2\end{aligned}$$



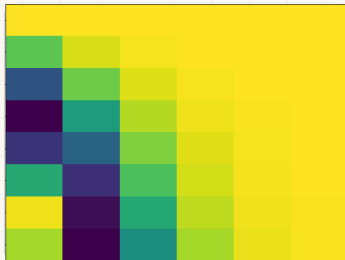
- We generalize this by using sin or cos instead of  $f(x) = \lfloor x \rfloor \bmod 2$ .

# Binary positional encoding

|         |   |   |   |   |   |   |
|---------|---|---|---|---|---|---|
| [CLS]   | 0 | 0 | 0 | 0 | 0 | 0 |
| because | 1 | 0 | 0 | 0 | 0 | 0 |
| it      | 0 | 1 | 0 | 0 | 0 | 0 |
| was     | 1 | 1 | 0 | 0 | 0 | 0 |
| two     | 0 | 0 | 1 | 0 | 0 | 0 |
| tired   | 1 | 0 | 1 | 0 | 0 | 0 |
| [EOS]   | 0 | 1 | 1 | 0 | 0 | 0 |

$BP$

$$\begin{aligned}BP_{ij} &= (i \div 2^j) \bmod 2 \\ &= \lfloor 2^{-j} i \rfloor \bmod 2\end{aligned}$$



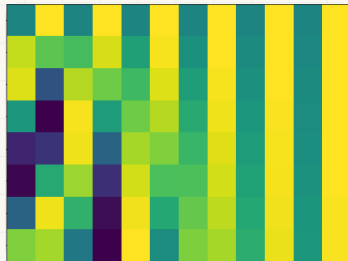
- We generalize this by using sin or cos instead of  $f(x) = \lfloor x \rfloor \bmod 2$ .

# Binary positional encoding

|         |   |   |   |   |   |   |
|---------|---|---|---|---|---|---|
| [CLS]   | 0 | 0 | 0 | 0 | 0 | 0 |
| because | 1 | 0 | 0 | 0 | 0 | 0 |
| it      | 0 | 1 | 0 | 0 | 0 | 0 |
| was     | 1 | 1 | 0 | 0 | 0 | 0 |
| two     | 0 | 0 | 1 | 0 | 0 | 0 |
| tired   | 1 | 0 | 1 | 0 | 0 | 0 |
| [EOS]   | 0 | 1 | 1 | 0 | 0 | 0 |

$BP$

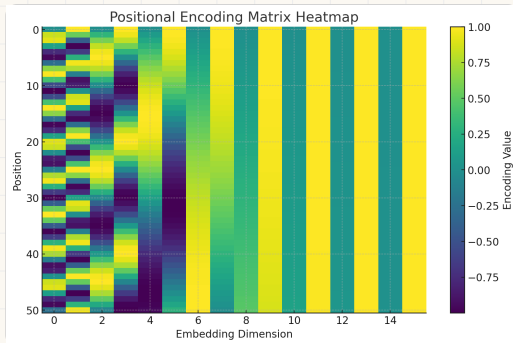
$$\begin{aligned}BP_{ij} &= (i \div 2^j) \bmod 2 \\ &= \lfloor 2^{-j} i \rfloor \bmod 2\end{aligned}$$



- We actually use both sin and cos and then interleave them.

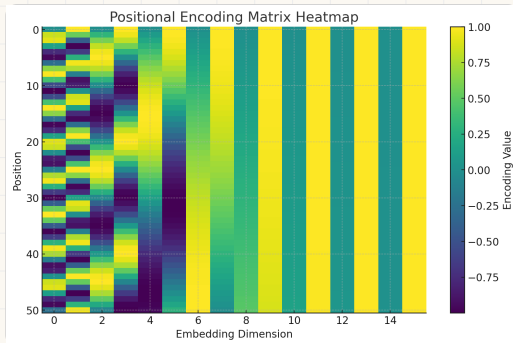
# Positional encodings

- The positional encoding matrix  $p$  has  $T$  rows and  $d$  columns.



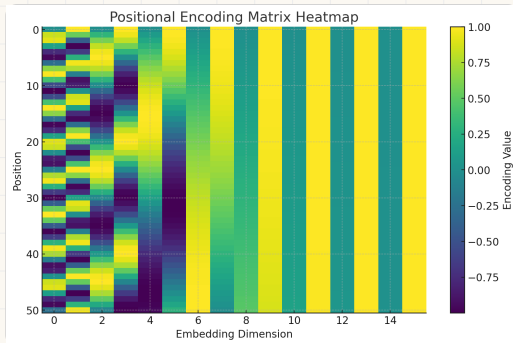
# Positional encodings

- The positional encoding matrix  $p$  has  $T$  rows and  $d$  columns.
- The value  $T$  is the maximum number of tokens in a sentence.



# Positional encodings

- The positional encoding matrix  $p$  has  $T$  rows and  $d$  columns.
- The value  $T$  is the maximum number of tokens in a sentence.
- The value  $d$  is the dimensionality of the positional embedding.



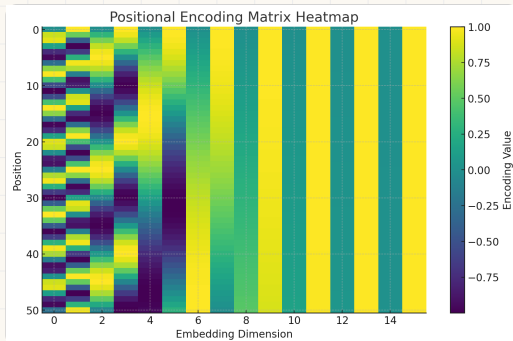
# Positional encodings

- The positional encoding matrix  $p$  has  $T$  rows and  $d$  columns.
- The value  $T$  is the maximum number of tokens in a sentence.
- The value  $d$  is the dimensionality of the positional embedding.
- The encoding of token  $i$  is defined through the formulas

$$p_{i,2j} = \sin(\alpha^{-2j} i) \text{ and}$$

$$p_{i,2j+1} = \cos(\alpha^{-2j} i),$$

for  $j \leq d/2$  and  $\alpha = 10^{4/d}$ .



# Why adding and not concatenating?

---

- Concatenation substantially increases the dimensionality of the embedding, increasing the number of parameters to be trained.

# Why adding and not concatenating?

---

- Concatenation substantially increases the dimensionality of the embedding, increasing the number of parameters to be trained.
- With sufficient dimensionality, since the positional embeddings are somewhat statistically independent from the token embeddings, both embeddings are somewhat orthogonal. Hence, the addition still preserves information about the two vectors.

# Why adding and not concatenating?

---

- Concatenation substantially increases the dimensionality of the embedding, increasing the number of parameters to be trained.
- With sufficient dimensionality, since the positional embeddings are somewhat statistically independent from the token embeddings, both embeddings are somewhat orthogonal. Hence, the addition still preserves information about the two vectors.
- Since most of the operations done in an attention mechanism are linear, the final embedding is somewhat the addition of the result of the operations applied to both the positional and the token embeddings.

6

*Section 6*

# Residual blocks

---

# The degradation problem

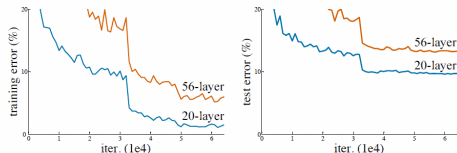


Figure 1. Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer “plain” networks. The deeper network has higher training error, and thus test error. Similar phenomena on ImageNet is presented in Fig. 4.

- When deeper networks are able to start converging, a *degradation problem* appeared. Training performance degrades rapidly. This is not because of overfitting.

Figure 1 of Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun. Deep Residual Learning for Image Recognition. CVPR 2016.

<https://arxiv.org/abs/1512.03385>

# The degradation problem

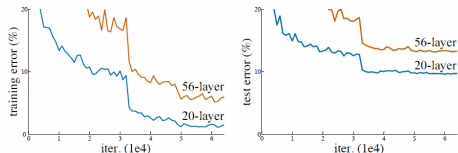


Figure 1. Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer “plain” networks. The deeper network has higher training error, and thus test error. Similar phenomena on ImageNet is presented in Fig. 4.

Figure 1 of Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun. Deep Residual Learning for Image Recognition. CVPR 2016.

<https://arxiv.org/abs/1512.03385>

- When deeper networks are able to start converging, a *degradation problem* appeared. Training performance degrades rapidly. This is not because of overfitting.
- The problem is mostly due to difficulties of information propagating, both forward and backward, through the layers in the network.

# Residual blocks

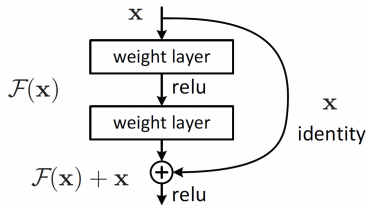


Figure 2. Residual learning: a building block.

Figure 2 of Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun. Deep Residual Learning for Image Recognition. CVPR 2016.

<https://arxiv.org/abs/1512.03385>

# Residual blocks

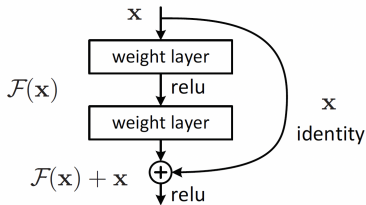


Figure 2. Residual learning: a building block.

- To address this problem, residual blocks were introduced. They work by adding the output of a layer to the output of a few layers ahead.

Figure 2 of Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun. Deep Residual Learning for Image Recognition. CVPR 2016.

<https://arxiv.org/abs/1512.03385>

7

*Section 7*

# Transformer architecture

---

# Transformer architecture

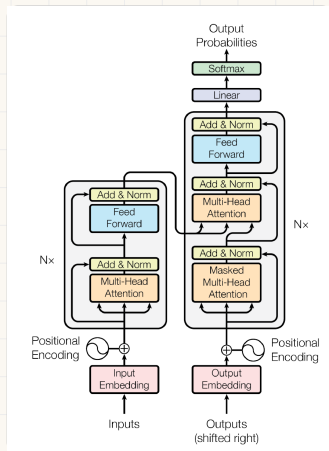


Figure 1 of

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser and Illia Polosukhin.

*Attention Is All You Need.*

Advances in Neural Information Processing Systems 30 (NeurIPS 2017).

<https://arxiv.org/abs/1706.03762>